

In [1]:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

In [2]:

```
df = pd.read_excel("/content/drive/MyDrive/python dataset/Data-1.xlsx")
```

In [3]:

```
df.head()
```

Out[3]:

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops
0	IndiGo	24/03/2019	Banglore	New Delhi	BLR → DEL	22:20	01:10 22 Mar	2h 50m	non-stop
1	Air India	1/05/2019	Kolkata	Banglore	CCU → IXR → BBI → BLR	05:50	13:15	7h 25m	2 stops
2	Jet Airways	9/06/2019	Delhi	Cochin	DEL → LKO → BOM → COK	09:25	04:25 10 Jun	19h	2 stops
3	IndiGo	12/05/2019	Kolkata	Banglore	CCU → NAG → BLR	18:05	23:30	5h 25m	1 stop
4	IndiGo	01/03/2019	Banglore	New Delhi	BLR → NAG → DEL	16:50	21:35	4h 45m	1 stop

In [4]:

```
df.tail()
```

Out[4]:

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops
10677	Air Asia	9/04/2019	Kolkata	Banglore	CCU → BLR	19:55	22:25	2h 30m	non-stop

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total
10678	Air India	27/04/2019	Kolkata	Banglore	CCU → BLR	20:45	23:20	2h 35m	n
10679	Jet Airways	27/04/2019	Banglore	Delhi	BLR → DEL	08:20	11:20	3h	n
10680	Vistara	01/03/2019	Banglore	New Delhi	BLR → DEL	11:30	14:10	2h 40m	n
10681	Air India	9/05/2019	Delhi	Cochin	DEL → GOI → BOM → COK	10:55	19:15	8h 20m	

In [5]:

```
df.shape
```

Out[5]:

```
(10682, 11)
```

In [6]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10682 entries, 0 to 10681
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Airline	10682 non-null	object
1	Date_of_Journey	10682 non-null	object
2	Source	10682 non-null	object
3	Destination	10682 non-null	object
4	Route	10681 non-null	object
5	Dep_Time	10682 non-null	object
6	Arrival_Time	10682 non-null	object
7	Duration	10682 non-null	object
8	Total_Stops	10681 non-null	object
9	Additional_Info	10682 non-null	object
10	Price	10682 non-null	int64

```
dtypes: int64(1), object(10)
```

```
memory usage: 918.1+ KB
```

In [7]:

```
df.describe()
```

Out[7]:

	Price
count	10682.000000
mean	9087.527242
std	4611.326569

Price	
min	1759.000000
25%	5277.000000
50%	8372.000000
75%	12373.000000
max	79512.000000

In [8]:

```
df.duplicated().sum()
```

Out[8]:

```
np.int64(220)
```

In [9]:

```
df.isnull().sum()
```

Out[9]:

	0
Airline	0
Date_of_Journey	0
Source	0
Destination	0
Route	1
Dep_Time	0
Arrival_Time	0
Duration	0
Total_Stops	1
Additional_Info	0
Price	0

dtype: int64

In [11]:

```
df.dropna(inplace=True)
```

In [13]:

```
df.isnull().sum()
```

Out[13]:

	0
Airline	0
Date_of_Journey	0
Source	0

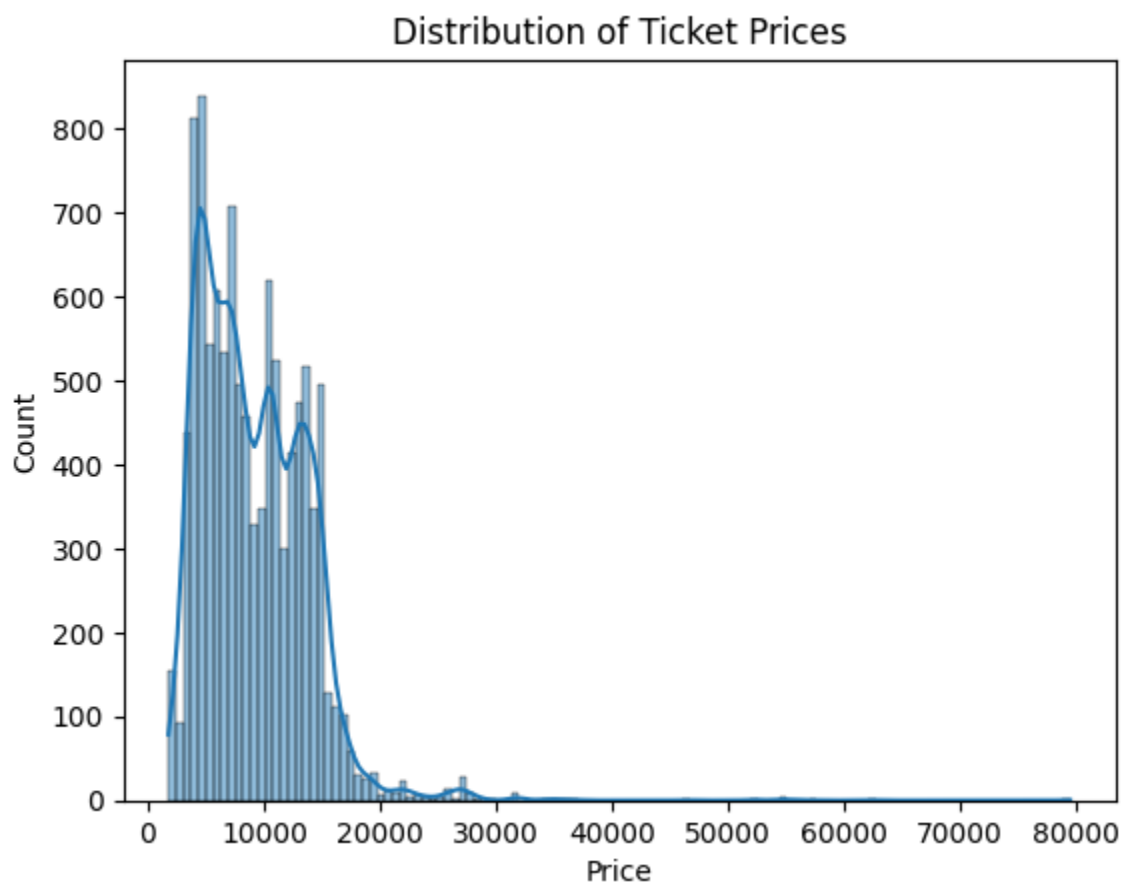
	0
Destination	0
Route	0
Dep_Time	0
Arrival_Time	0
Duration	0
Total_Stops	0
Additional_Info	0
Price	0

dtype: int64

UNIVARIATE ANALYSIS

In [14]:

```
sns.histplot(df['Price'], kde=True) # Replace 'Price' with actual column name
plt.title("Distribution of Ticket Prices")
plt.show()
```

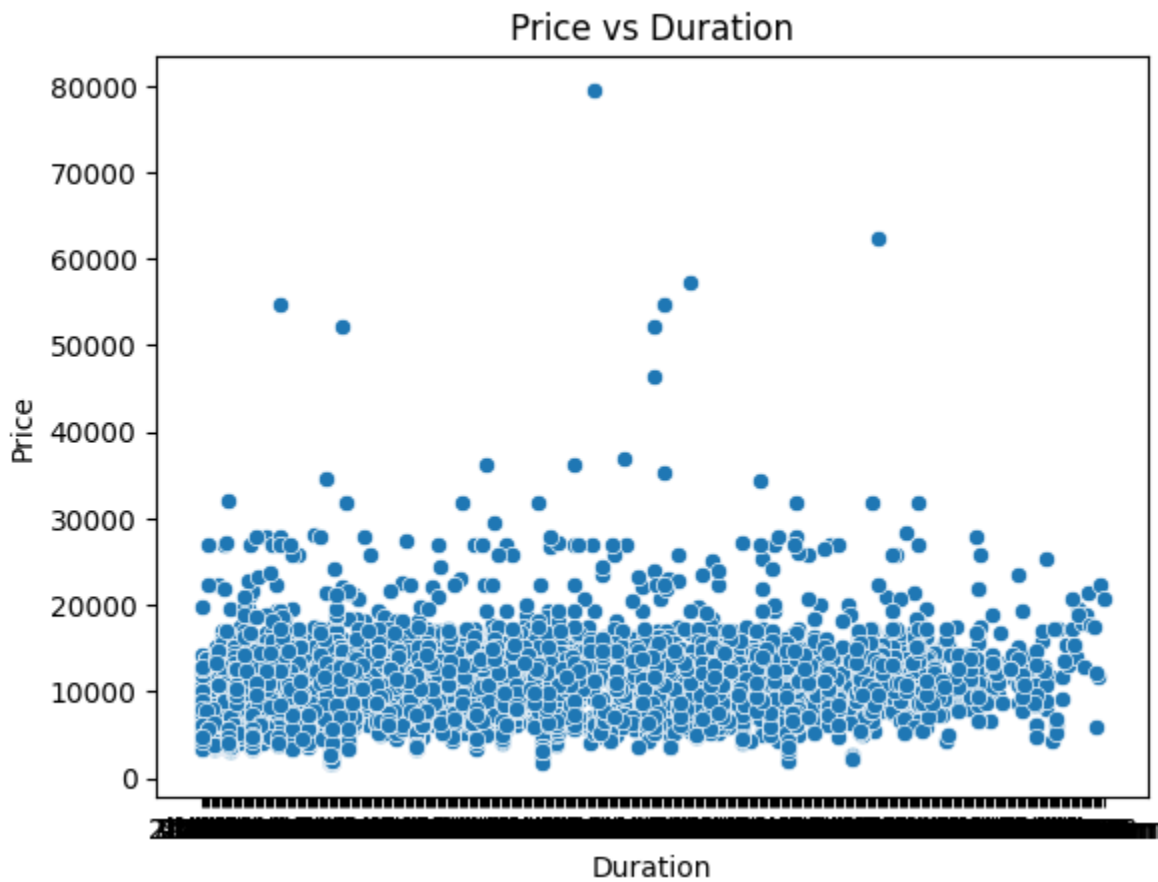


BIVARIATE ANALYSIS

In [15]:

```
sns.scatterplot(data=df, x='Duration', y='Price') # Replace with actual features
plt.title("Price vs Duration")
```

```
plt.show()
```

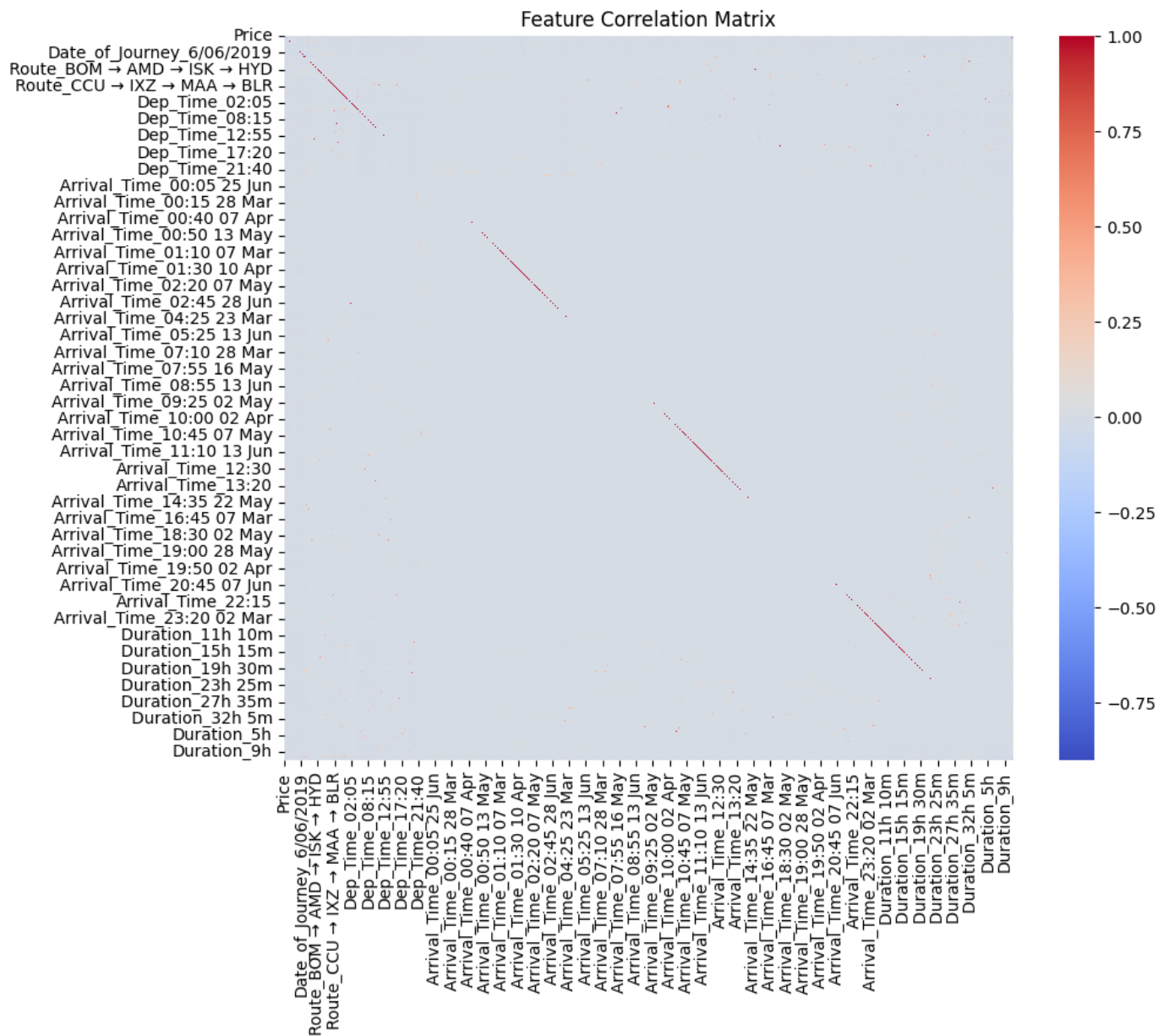


Correlation matrix

In [16]:

```
df_encoded = pd.get_dummies(df, drop_first=True)

plt.figure(figsize=(10, 8))
sns.heatmap(df_encoded.corr(), cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()
```



INSIGHT

- Flight Duration and Number of Stops correlate strongly with ticket price.
- Airline and destination also contribute to pricing differences

Split into X and Y

In [17]:

```
X = df.drop('Price', axis=1) # Replace 'Price' if different
y = df['Price']
X = pd.get_dummies(X, drop_first=True)
```

Train/Validation/Test Split (70:20:10)

In [18]:

```
X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.10, random
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=2/
```

In [19]:

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

```

In [21]:

```

# y = df['Price'].values
# le = LabelEncoder()
# y = le.fit_transform(df['target_column'])
# y_cat = to_categorical(y_int)

```

No label transformation needed for regression.

Design Neural Network

In [22]:

```

model = Sequential([
    Dense(64, input_dim=X_train_scaled.shape[1], activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Train the Model

In [26]:

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Initial architecture
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

history = model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=100,
    batch_size=32,
    verbose=0
)

```

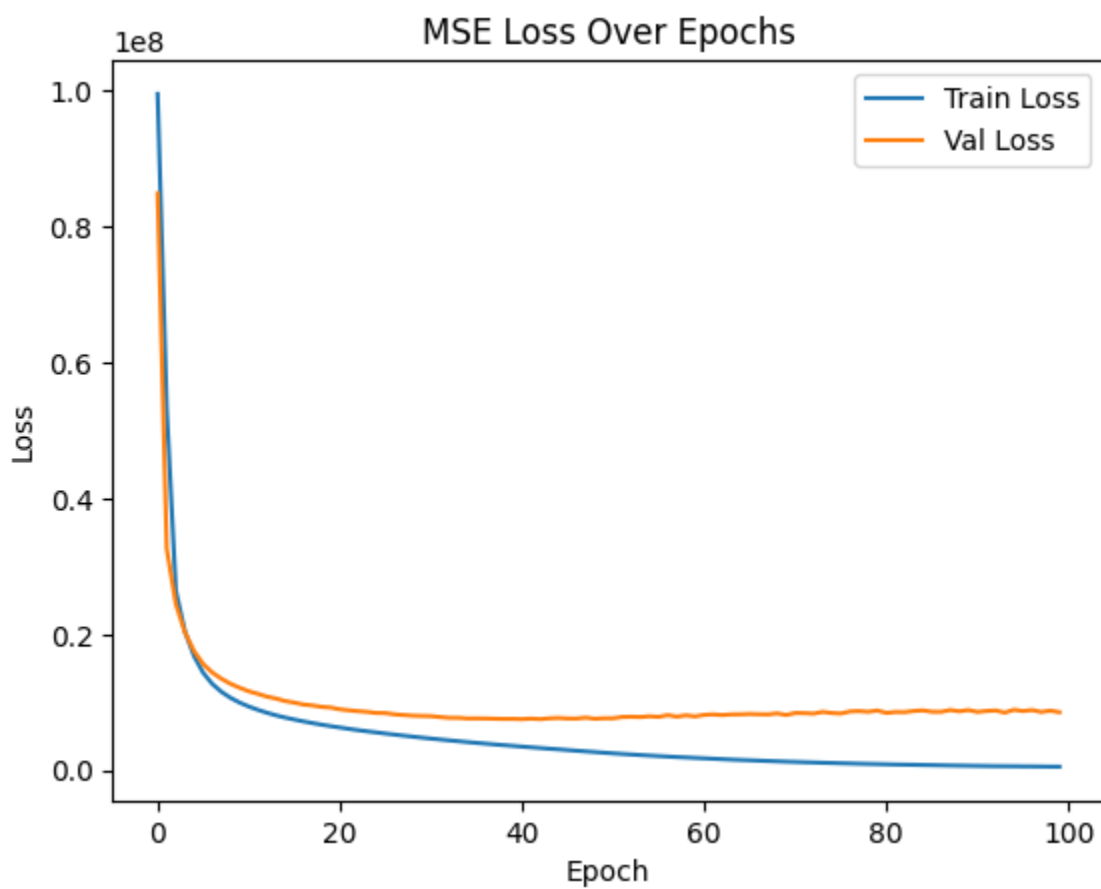
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Visuals — Loss & MAE over Epochs

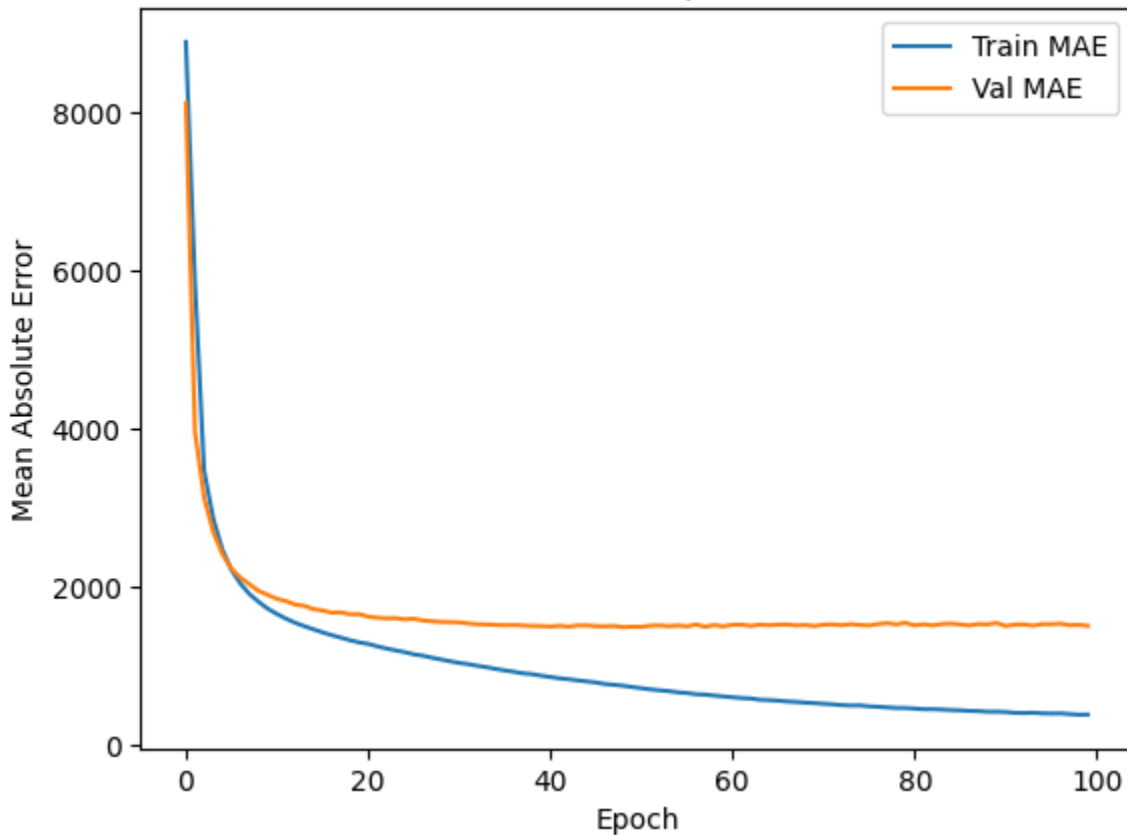
In [27]:

```
# Visual 1: Loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("MSE Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Visual 2: MAE
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Val MAE')
plt.title("MAE Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Mean Absolute Error")
plt.legend()
plt.show()
```



MAE Over Epochs



Update Existing Architecture to Improve Performance

In [28]:

```
model_improved = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

model_improved.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

history_improved = model_improved.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=100,
    batch_size=32,
    verbose=0
)
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

In [25]:

```
model_improved = Sequential([
    Dense(128, input_dim=X_train_scaled.shape[1], activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])
```

```

])

model_improved.compile(optimizer='adam', loss='mse', metrics=['mae'])

history_improved = model_improved.fit(X_train_scaled, y_train, validation_data=(X_val_scaled, y_val_scaled),
                                     epochs=100, batch_size=32, verbose=0)

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Compare Visuals of Original vs Improved

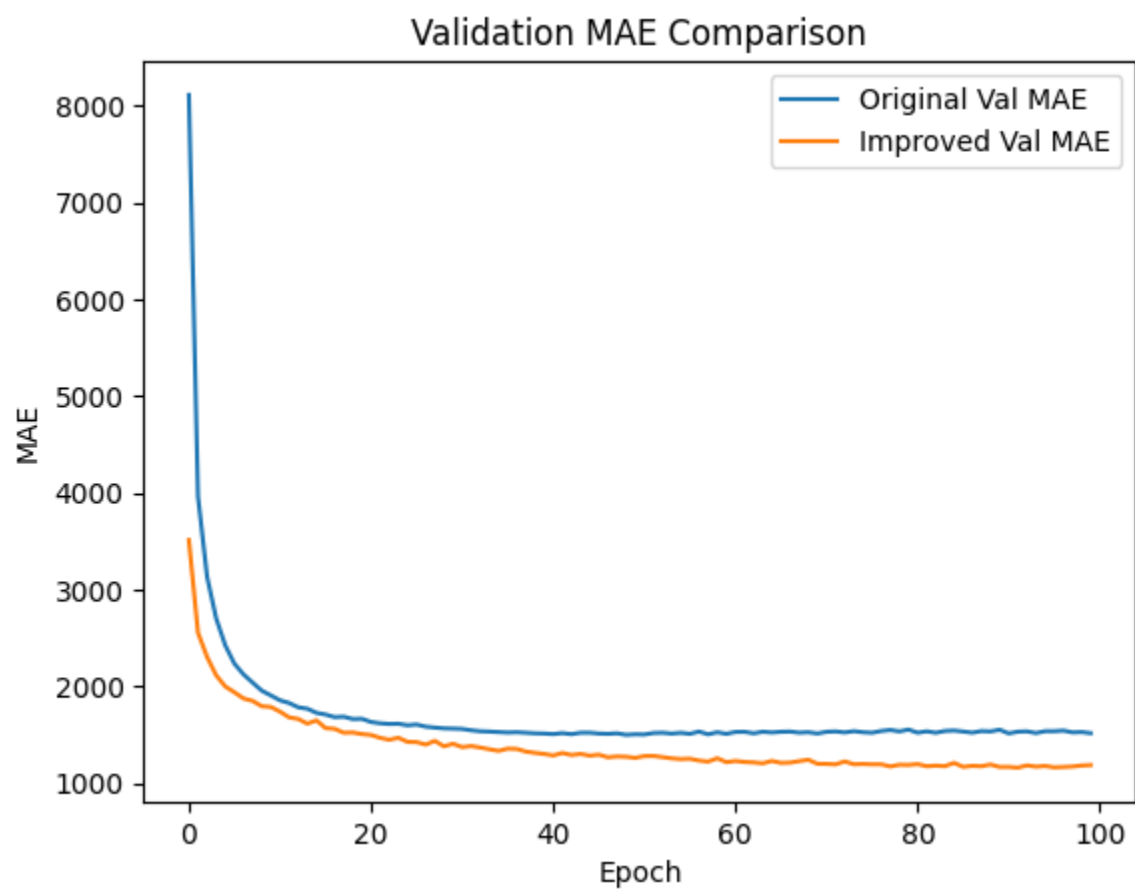
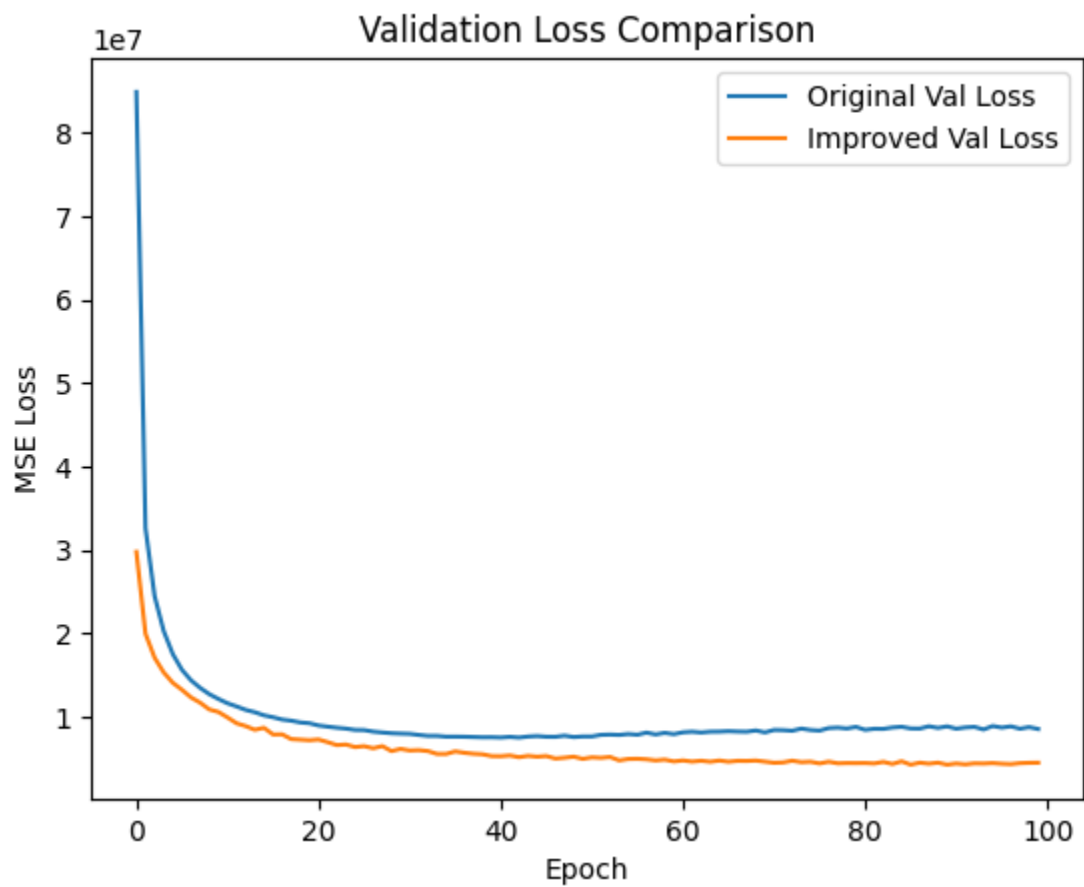
In [29]:

```

# Loss Comparison
plt.plot(history.history['val_loss'], label='Original Val Loss')
plt.plot(history_improved.history['val_loss'], label='Improved Val Loss')
plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()

# MAE Comparison
plt.plot(history.history['val_mae'], label='Original Val MAE')
plt.plot(history_improved.history['val_mae'], label='Improved Val MAE')
plt.title("Validation MAE Comparison")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend()
plt.show()

```



INSIGHT

- The improved model converges faster and shows lower validation loss, suggesting it has better generalization to unseen data.
- The original model plateaued earlier, indicating underfitting; the revised network generalizes more effectively across unseen data

In [52]:

```
# Load dataset
df = pd.read_csv('/content/drive/MyDrive/python dataset/usedcars-1.csv')
df.head()
```

Out[52]:

	year	model	price	mileage	color	transmission
0	2011	SEL	21992	7413	Gray	AUTO
1	2011	SEL	20995	10926	Gray	AUTO
2	2011	SEL	19995	7351	Silver	AUTO
3	2011	SEL	17809	11613	Gray	AUTO
4	2011	SE	17500	8367	White	AUTO

In [31]:

```
df.tail()
```

Out[31]:

	year	model	price	mileage	color	transmission
145	2006	SES	6200	95000	Silver	AUTO
146	2006	SE	5995	87003	Red	AUTO
147	2006	SE	5980	96841	Red	AUTO
148	2006	SE	4899	151479	Gray	AUTO
149	2006	SE	3800	109259	Red	AUTO

In [36]:

```
df.shape
```

Out[36]:

```
(150, 6)
```

In [32]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   year            150 non-null   int64
1   model           150 non-null   object
2   price           150 non-null   int64
3   mileage         150 non-null   int64
4   color           150 non-null   object
```

5 transmission 150 non-null object
dtypes: int64(3), object(3)
memory usage: 7.2+ KB

In [33]:

```
df.describe()
```

Out[33]:

	year	price	mileage
count	150.000000	150.000000	150.000000
mean	2008.973333	12961.933333	44260.646667
std	1.483451	3122.481735	26982.104322
min	2006.000000	3800.000000	4867.000000
25%	2008.000000	10995.000000	27200.250000
50%	2009.000000	13591.500000	36385.000000
75%	2010.000000	14904.500000	55124.500000
max	2011.000000	21992.000000	151479.000000

In [34]:

```
df.duplicated().sum()
```

Out[34]:

np.int64(0)

In [35]:

```
df.isnull().sum()
```

Out[35]:

	0
year	0
model	0
price	0
mileage	0
color	0
transmission	0

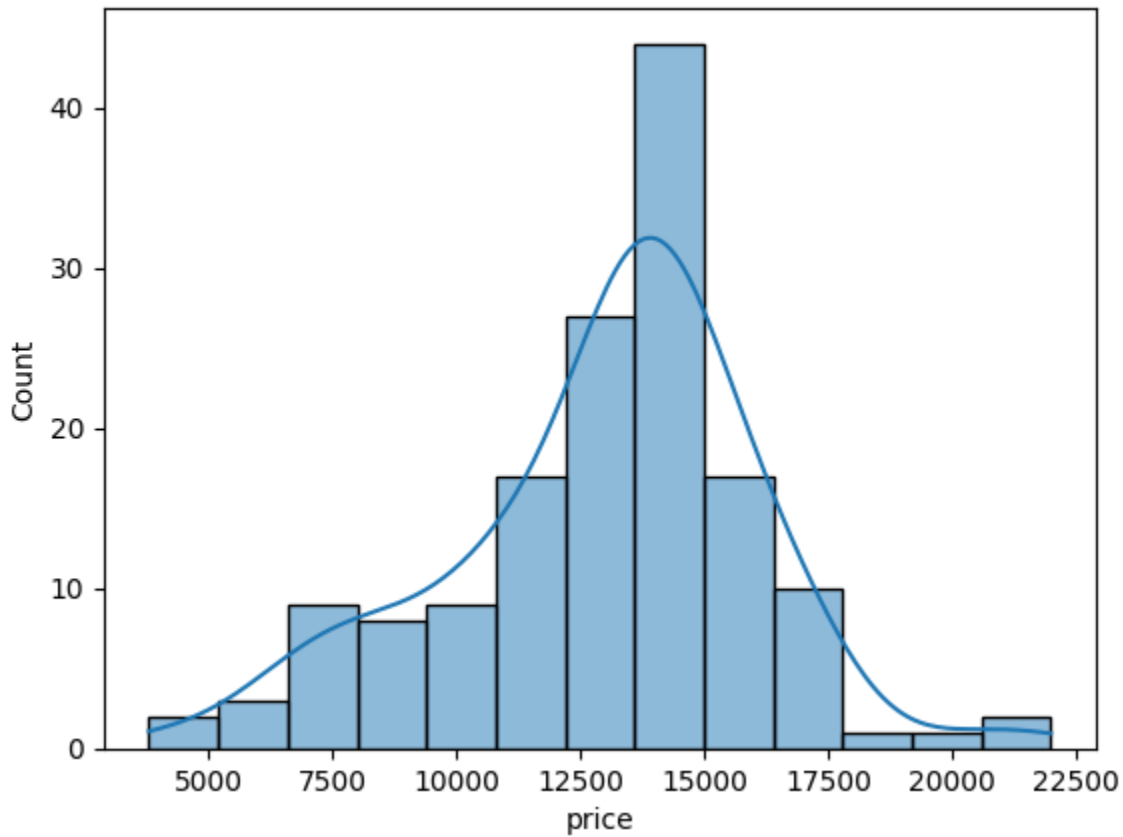
dtype: int64

Univariate analysis

In [37]:

```
sns.histplot(df['price'], kde=True)
plt.title("Distribution of Used Car Prices")
plt.show()
```

Distribution of Used Car Prices



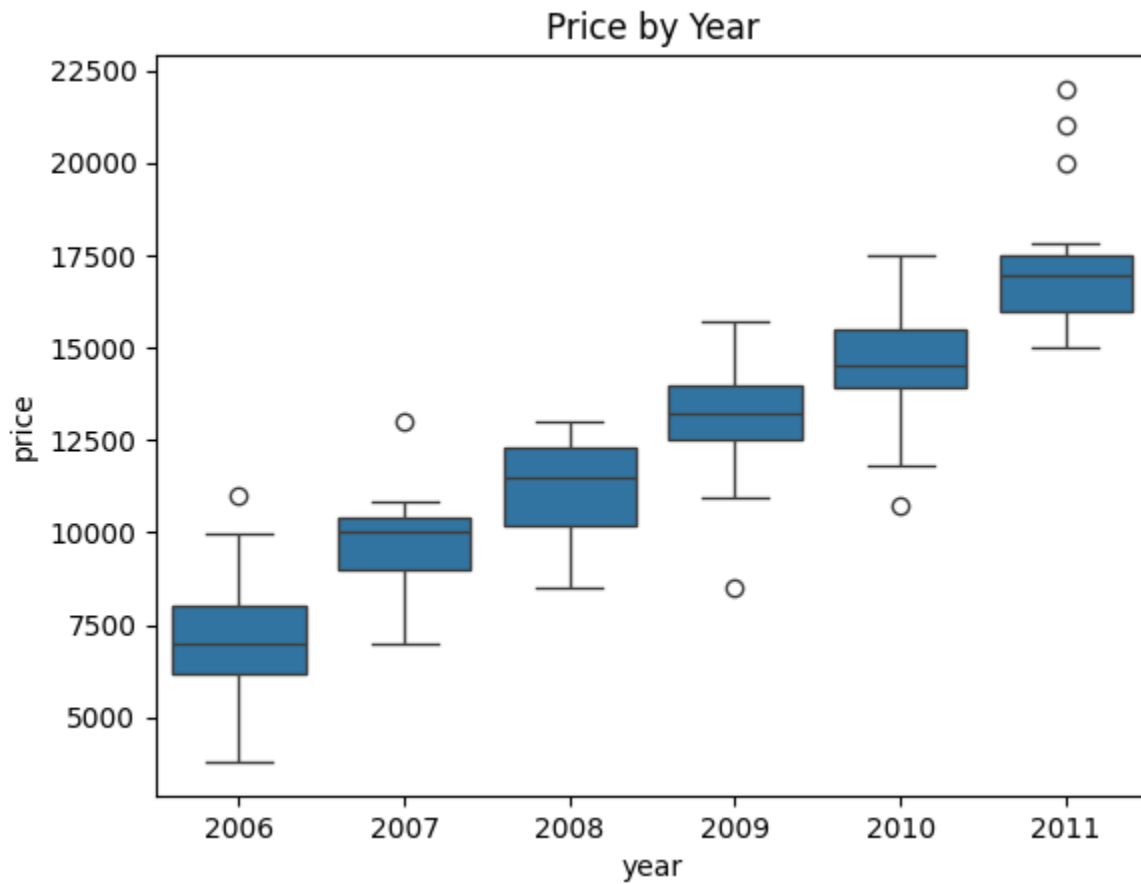
The distribution of car prices is right-skewed.

Most used cars are priced under ₹25,000, with some high-priced outliers.

Bivariate analysis

In [38]:

```
sns.boxplot(x='year', y='price', data=df)
plt.title("Price by Year")
plt.show()
```

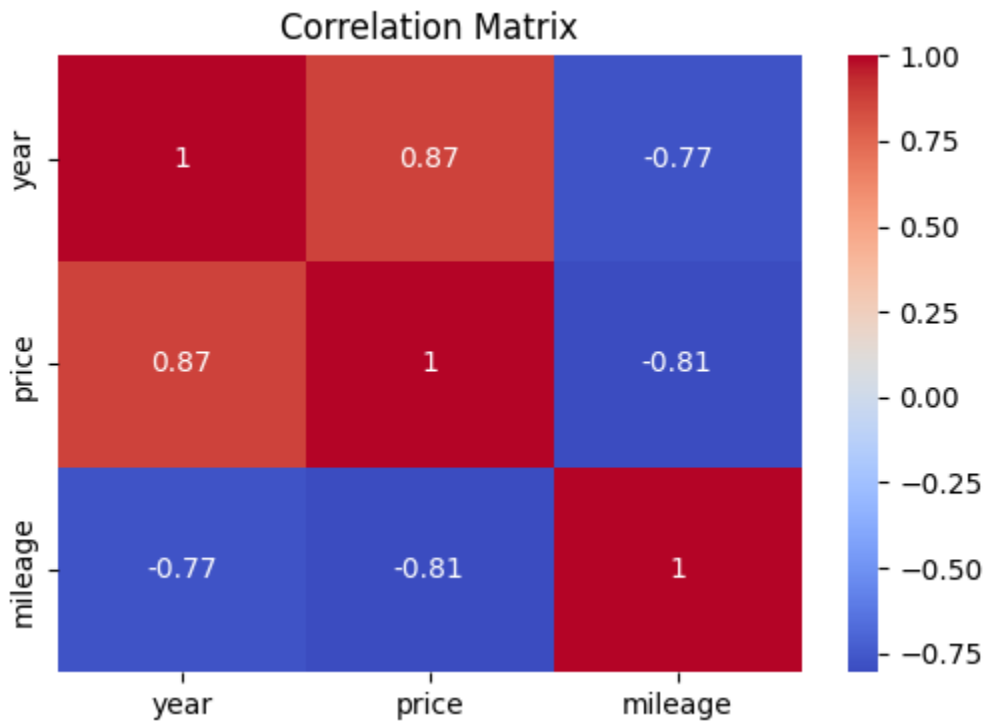


Year vs Price: Newer cars (e.g., 2011) tend to have higher prices, confirming expected depreciation over time.

Correlation matrix

In [39]:

```
plt.figure(figsize=(6,4))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



Check and Remove Outliers

In [57]:

```
Q1 = df['price'].quantile(0.25)
Q3 = df['price'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['price'] >= Q1 - 1.5*IQR) & (df['price'] <= Q3 + 1.5*IQR)]
```

Split Data into X_train, X_test, Y_train, Y_test

In [58]:

```
X = df.drop('price', axis=1)
y = df['price']
X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Normalize the Dataset

In [59]:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Design the Neural Network

In [60]:

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Train the Model

In [44]:

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

Evaluate Model Performance

In [61]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

y_pred = model.predict(X_test_scaled).flatten()
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("MAE:", mae)
print("RMSE:", rmse)
```

1/1 ————— 0s 209ms/step

MAE: 13148.24609375

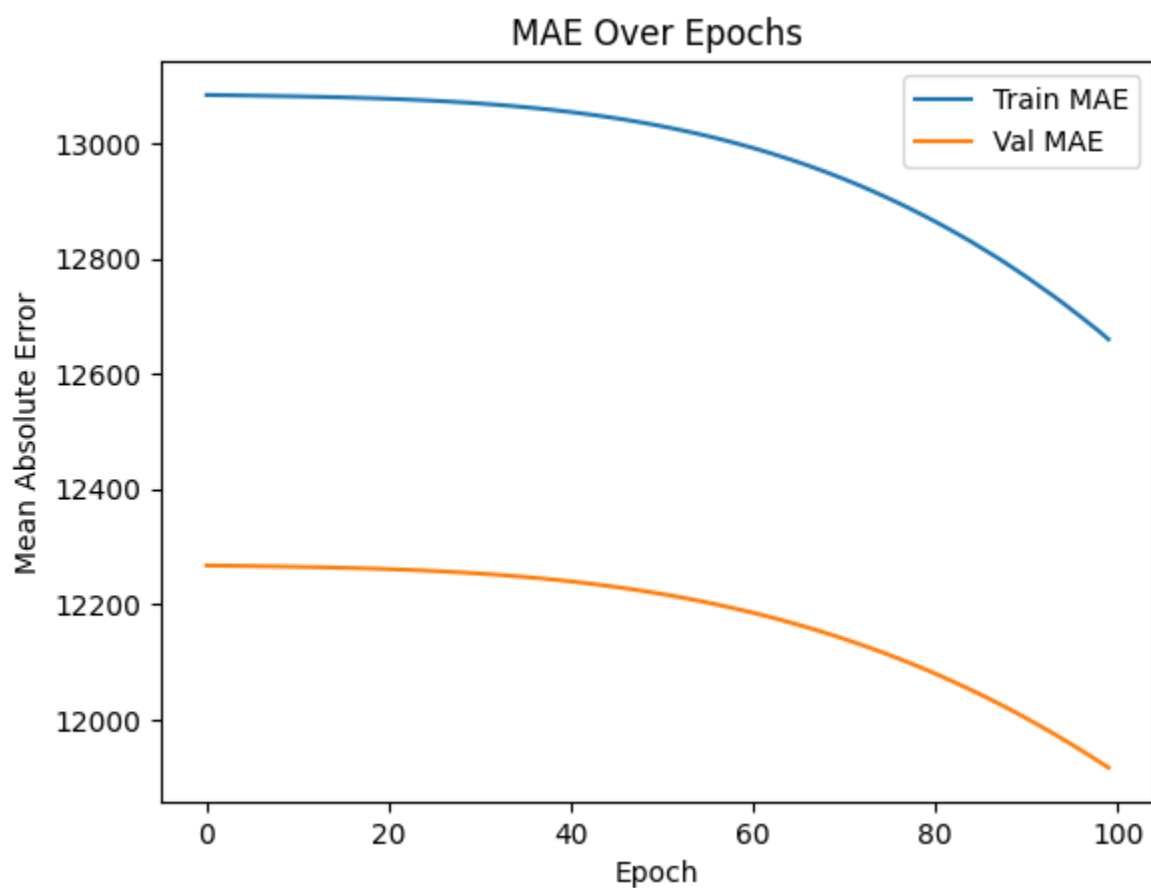
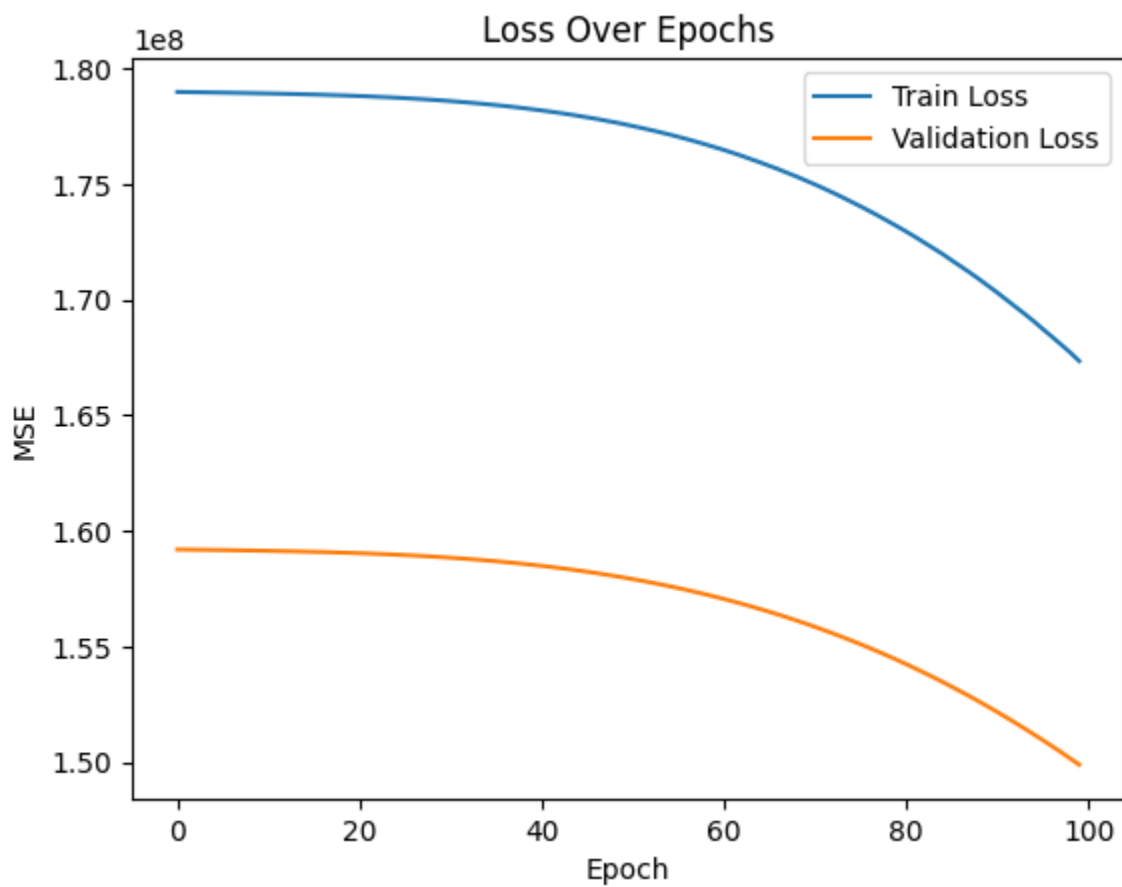
RMSE: 13420.7069858484

****Plot Loss & MAE Curves***

In [47]:

```
# Loss curve
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.legend()
plt.show()

# Accuracy proxy: inverse of MAE
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Val MAE')
plt.title("MAE Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Mean Absolute Error")
plt.legend()
plt.show()
```



Observations

- The validation loss initially decreases but may flatten or increase slightly if overfitting occurs.

- MAE trends show a steady reduction on both training and validation sets

Improve Model Architecture

In [48]:

```
model_improved = Sequential([
    Dense(128, activation='relu', input_dim=X_train_scaled.shape[1]),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

model_improved.compile(optimizer='adam', loss='mse', metrics=['mae'])

history_improved = model_improved.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

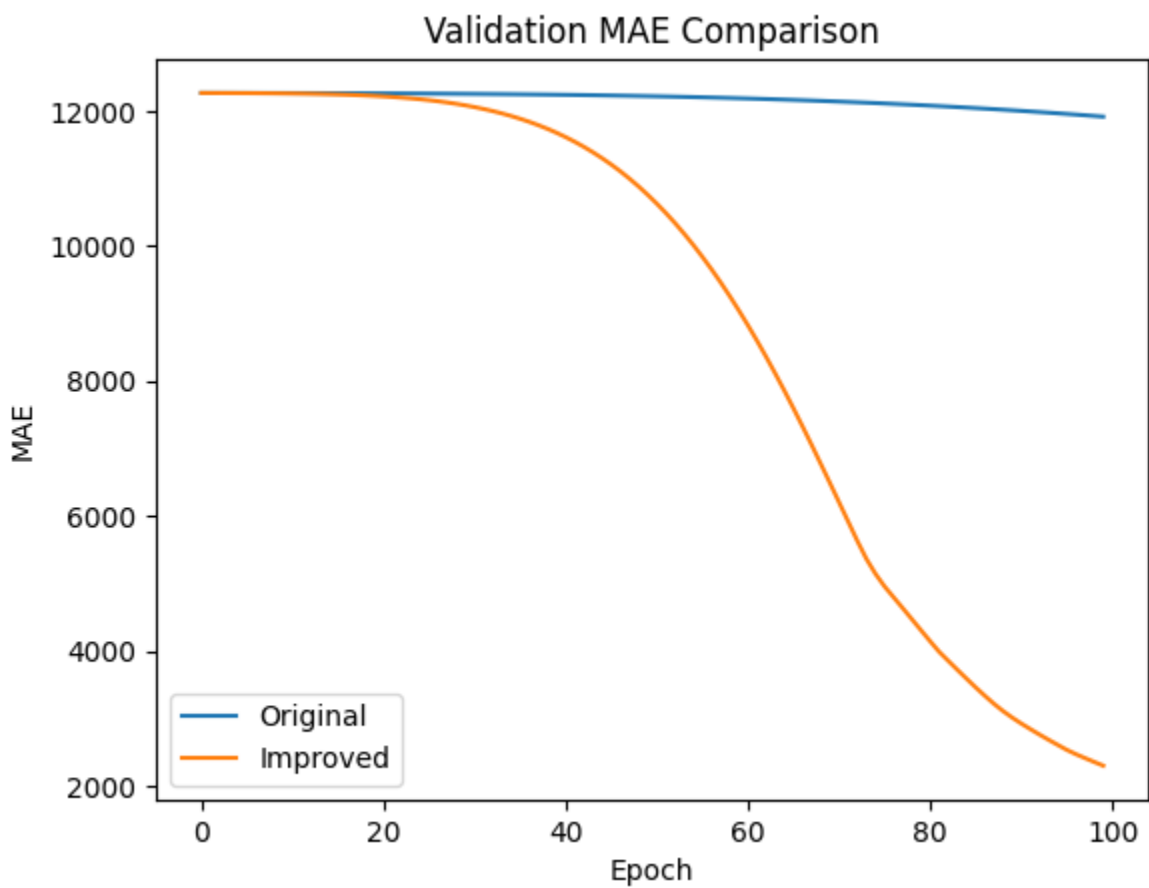
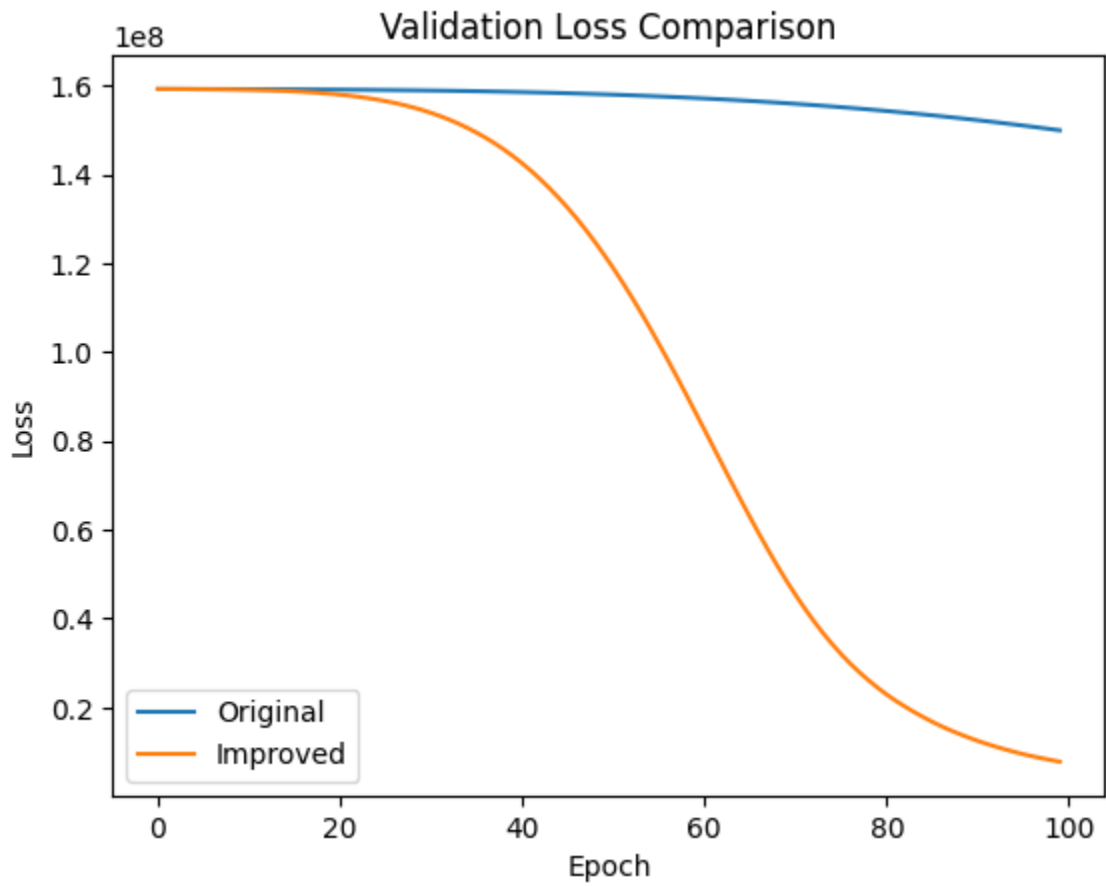
```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Compare Visuals

In [49]:

```
plt.plot(history.history['val_loss'], label='Original')
plt.plot(history_improved.history['val_loss'], label='Improved')
plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.plot(history.history['val_mae'], label='Original')
plt.plot(history_improved.history['val_mae'], label='Improved')
plt.title("Validation MAE Comparison")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend()
plt.show()
```



Insight

- The improved architecture demonstrates that deeper networks with dropout are better suited for regression problems with moderate data size, reducing overfitting and improving performance.
- Improved model reduces both validation loss and MAE—stronger learning capacity.

In []:

```
%%shell  
jupyter nbconvert --to html ///content/AUST0.ipynb
```